

CONTENTS

Práctico 4 (Memorias ROM) — Arquitectura de Computadoras	
FING UdelaR · Resolución completa y verificada	
Preguntas teóricas	
Ejercicio 1 — Combinación de ROMs de 8K×8	
Ejercicio 2 — Traductor ASCII mayúsculas↔minúsculas (en OpenFing)	
Ejercicio 3 — Shift aritmético a la derecha	
Ejercicio 4 — Impresora de matriz de puntos 8×8	
Ejercicio 5 — ROM de 32K×16 como base	
Ejercicio 6 — ROM traductora de punto flotante 16→32 bits (sin solución de referencia)	
Ejercicio 7 — ROM de 8 bits maximizando capacidad a partir de 4 chips distintos (en OpenFing)	
Ejercicio 8 — ALU simulada con ROM (Examen Julio 2022, sin solución de referencia)	
Resumen de técnicas usadas en este práctico	

Práctico 4 (Memorias ROM) — Arquitectura de Computadoras

FING UDELAR · RESOLUCIÓN COMPLETA Y VERIFICADA

⚠ **Nota general:** la solución de referencia del curso (`Ejercicios Resueltos.pdf`) no incluye los Ejercicios 6 y 8 (confirmado también en `plan_de_estudio.md`, fila del Práctico 4: “Ej. 6 y 8 sin solución”). Ambos se resuelven acá desde cero, con verificación numérica independiente (conversión IEEE754 verificada bit a bit contra `struct.pack` de Python). Además, se encontró y corrigió un bug real de precedencia de operadores en el código C del Ejercicio 3 de la solución de referencia — ver nota ⚠ en ese ejercicio.

Preguntas teóricas

(a) Propósito de CS y OE.

- **CS (Chip Select):** habilita o deshabilita el chip **completo**. Con CS desactivado, el chip no responde a ninguna otra señal (direcciones, OE): es como si no estuviera conectado al circuito.
- **OE (Output Enable):** con el chip ya habilitado (CS activo), controla si el dato leído se **efectivamente coloca** en el bus de salida, o si las salidas quedan en **alta impedancia** (tercer estado). Un chip puede estar “encendido” (CS=1) pero “en silencio” (OE=0).
- Distinción clave: CS decide *qué chip participa*; OE decide *si ese chip habla en el bus en este instante*. Esta separación es la que permite que varios chips compartan un mismo bus de datos (Ejercicios 1, 5, 7, 8).

(b) ROM para implementar una función lógica arbitraria. El valor de cualquier función lógica $F(x_0, \dots, x_{n-1})$ depende únicamente del valor de sus n entradas. Si se conecta cada variable x_i a una línea de dirección A_i de una ROM, entonces cada una de las 2^n direcciones posibles corresponde exactamente a una combinación de entrada distinta. Basta con grabar, en la dirección $A_0A_1\dots A_{n-1} = x_0x_1\dots x_{n-1}$, el valor $F(x_0, \dots, x_{n-1})$ — es decir, volcar la tabla de verdad completa de F en el contenido de la ROM. Una función de salida de m bits requiere m líneas de datos (una ROM de $2^n \times m$).

(c) ¿Combinatorio o secuencial? Combinatorio. La salida depende únicamente del valor *actual* de la dirección de entrada — no existe ningún elemento de memoria de estado interno que dependa de valores pasados. Dos lecturas consecutivas a la misma dirección siempre devuel-

ven el mismo dato, sin importar qué se haya leído antes (a diferencia de un circuito secuencial, cuya salida depende también del estado interno acumulado).

(d) Tercer estado. Es un tercer valor de salida (además de 0 y 1) en el que la salida del circuito queda **eléctricamente desconectada** del resto del circuito (alta impedancia): no impone ni 0 ni 1 sobre la línea a la que está conectada, y no conduce corriente. Su utilidad central es permitir que **múltiples chips compartan el mismo bus de datos**: en todo instante, a lo sumo un chip tiene su salida activa (no en alta impedancia); todos los demás “se apartan” eléctricamente del bus, evitando que dos chips impongan valores distintos sobre la misma línea al mismo tiempo (lo cual sería un cortocircuito lógico).

(e) Variantes de ROM que se ajustan estrictamente a la sigla. De las variantes vistas (ROM de máscara, PROM, EPROM, EEPROM), la única que se ajusta **estrictamente** al significado literal “Read-Only” es la **PROM ya programada**: permite grabar el contenido una única vez después de fabricado el chip, pero luego no admite ni borrado ni reescritura. Las demás variantes (EPROM, borrable por luz UV; EEPROM, borrable eléctricamente) sí permiten reescritura, por lo que en sentido estricto no son “de solo lectura”, aunque se empleen como tales en operación normal.

(f) Problema del OR cableado entre ROMs. Sí, hay un problema real. Las compuertas (y las propias salidas de las ROMs) tienen **retardos de propagación** no nulos y no perfectamente iguales entre chips. Si se combinan las salidas de datos de varias ROMs con una compuerta OR (en vez de con CS + tercer estado), durante el breve intervalo en que las señales de distintos chips están conmutando (algunas ya cambiaron, otras no), el bus puede mostrar transitoriamente un valor **incorrecto o fluctuante** que no corresponde a ningún dato válido. La solución correcta —usada en todo el práctico— es garantizar por diseño que **como máximo un chip tenga su salida activa** en cada instante (vía CS/decodificador) y unir las salidas directamente al bus compartido (tercer estado), nunca a través de una compuerta lógica.

Ejercicio 1 — Combinación de ROMs de 8K×8

Chip disponible: **8K×8**. $N=8K=2^{13}=8192$ palabras (13 líneas de dirección), $M=8$ (8 líneas de datos).

(a) Fabricar una ROM de 32K×8. Solo cambia N (mismo ancho $M=8$ en ambos) → caso “más cantidad de palabras” (decodificador + CS).

- $N_{\text{nueva}} = 32K = 2^{15} = 32768 \rightarrow 15$ líneas de dirección totales.
- Cantidad de chips = $32K/8K = 32768/8192 = 4$ chips.

- Líneas extra = $15 - 13 = 2$ líneas (las 2 más significativas, A_{13}, A_{14}) → van a un **decodificador 2→4**, cuyas 4 salidas activan el CS de cada uno de los 4 chips (una sola a la vez).
- Las 13 líneas de dirección menos significativas ($A_0..A_{12}$) van **en paralelo** a los 4 chips.
- OE se deja activo en los 4 chips (o común); como CS garantiza que solo uno “existe” en cada instante, las 4 salidas $D_0..D_7$ se conectan **directamente** al mismo bus (tercer estado en los demás).

Verificación: $\log_2(32768)=15$, $\log_2(8192)=13$, $15-13=2$, $2^2=4$ chips ✓. Capacidad:

$4 \times (8192 \times 8) = 262144$ bits = 32768×8 ✓ (coincide exactamente).

(b) Fabricar una ROM de 16K×16. Cambian **ambos** ejes: N sube (8K→16K) y M sube (8→16) → caso general (grilla de bancos × paralelo).

- **Eje N:** $16K/8K = 16384/8192 = 2$ bancos. Líneas extra = $\log_2(16384) - 13 = 14 - 13 = 1$ línea (A_{13}) → decodificador 1→2 (implementable con un simple inversor + 2 AND, como en el diagrama de la solución del curso) que selecciona el CS de cada banco.
- **Eje M:** $16/8 = 2$ ROMs en paralelo por banco (mismas direcciones y mismo CS/OE dentro del banco; salidas concatenadas: una da $D_0..D_7$, la otra $D_8..D_{15}$).
- **Total de chips** = 2 bancos × 2 en paralelo = **4 ROMs de 8K×8**.

Verificación: capacidad necesaria = $16384 \times 16 = 262144$ bits. Capacidad de 4 chips de $8K \times 8 = 4 \times 8192 \times 8 = 262144$ bits ✓. Líneas de dirección: $\log_2(16384)=14 = 13$ (internas de cada chip) + 1 (decodificador) ✓.

Ejercicio 2 — Traductor ASCII mayúsculas↔minúsculas (en OpenFing)

(a) Tamaño y organización. El alfabeto ASCII estándar tiene $2^7=128$ caracteres distintos (7 bits). Como la transformación (mayúscula↔minúscula, resto sin cambios) depende únicamente del carácter de entrada y produce otro carácter de 7 bits, se necesita:

- **Entradas:** $A_0..A_6$ (7 líneas) = el código ASCII del carácter de entrada.
- **Salidas:** $D_0..D_6$ (7 líneas) = el código ASCII del carácter transformado.
- **Tamaño:** $2^7 \times 7 = \text{ROM de } 128 \times 7$.
- Más CS y OE.

(b) Programa en C que genera el contenido.

```

void cargar_rom(char* rom[]) {
    for (int i = 0; i < 128; i++) {           // i recorre las 2^7 = 128 direcciones
        if (i >= 0x41 && i <= 0x5A) {        // 'A'-'Z' -> minuscula
            rom[i] = i + ('a' - 'A');
        } else if (i >= 0x61 && i <= 0x7A) {  // 'a'-'z' -> mayuscula
            rom[i] = i - ('a' - 'A');
        } else {
            rom[i] = i;                      // el resto, sin cambios
        }
    }
}

```

`i` cumple triple función: contador del `for`, dirección de la ROM (`rom[i]`) y valor del carácter ASCII de entrada evaluado por las condiciones — deben coincidir siempre.

Ejercicio 3 — Shift aritmético a la derecha

(a) Entradas, salidas, tamaño.

- Entrada de dato: 1 byte a desplazar → 8 bits ($A_3..A_{10}$, por ejemplo).
- Entrada de control: 3 bits que indican cuánto desplazar (0 a 7 posiciones) → ($A_0..A_2$).
- Total entradas = $8+3 = 11$ **líneas de dirección** → $2^{11} = 2048 = 2K$ direcciones.
- Salida: el byte ya desplazado → **8 líneas de datos**.
- **Tamaño: ROM de $2K \times 8$.**

Verificación: $2^{11}=2048=2 \times 1024=2K \checkmark$.

(b) Programa que genera el contenido — con corrección respecto a la solución de referencia.

```

void cargar_rom(unsigned char rom[256][8]) {
    for (int i = 0; i < 8; i++) {           // i = cantidad de bits a desplazar (0..7)
        for (int j = 0; j < 256; j++) {    // j = byte a desplazar (0..255)
            if ((j & 0x80) != 0) {         // bit de signo = 1 -> j es "negativo"
                rom[j][i] = (unsigned char)((j >> i) | (0xFF << (8 - i)));
            } else {
                rom[j][i] = (unsigned char)(j >> i);
            }
        }
    }
}

```

⚠ **Corrección de tolerancia cero sobre la solución de referencia del curso:** el código original de `Ejercicios Resueltos.pdf` escribe la condición como `if (j & 0x80 == 1)`. En C, el operador `==` tiene **mayor precedencia** que `&`, por lo que esa expresión se evalúa como `j & (0x80 == 1) = j & 0 = 0` — es decir, la rama que inserta los “1” a la izquierda para números negativos **nunca se ejecuta**, sin importar el valor de `j`. Es un bug real de precedencia de operadores, un error clásico de C. La condición correcta es `(j & 0x80) != 0` (equivalentemente, alcanza con `if (j & 0x80)`).

Verificación de la fórmula (independiente del bug de precedencia): para $j=0x80$ (10000000, es decir, -128 en complemento a 2) e $i=1$ (desplazar 1 bit), el shift aritmético correcto da $11000000 = 0xC0$ (-64). Con la fórmula: `(0x80 >> 1) | (0xFF << 7) = 0x40 | 0x7F80 = 0x7FC0`, que truncado a 8 bits (byte) da `0xC0` ✓. Para $i=0$ (sin desplazamiento), `(0xFF << 8)` trunca a 0 al guardarse en un byte, dejando `rom[j][0] = j` sin cambios, como corresponde ✓.

Ejercicio 4 — Impresora de matriz de puntos 8×8

⚠ **Nota de transparencia:** la Figura 1 del enunciado (patrón de puntos de la letra “A”) se extrajo como tabla de texto y no como imagen nítida; el patrón general (forma de “A”: vértice arriba, línea central horizontal, dos “patas” verticales en los extremos) se puede reconstruir con confianza, pero no hace falta el contenido punto-por-punto para responder (a) y (b) — solo se pide el **método** de direccionamiento y construcción, no volcar la tabla completa. Si se necesitara el contenido exacto para otro ejercicio, conviene verificar la figura contra el PDF original.

(a) Entradas, salidas, tamaño y organización. La clave es notar que el enunciado dice explícitamente **“la impresora imprime una columna de puntos por vez”** — es decir, en cada acceso a la ROM se pide **una sola columna** (8 bits: qué filas están encendidas), no las 8 columnas simultáneamente. Esto sugiere el mismo patrón que en el Ejercicio 3 (dato + selector, concatenados como dirección):

- **Carácter a imprimir:** código ASCII extendido, 8 bits ($A_3..A_{10}$).
- **Columna a imprimir:** 3 bits, ya que hay 8 columnas por carácter, 0 a 7 ($A_0..A_2$).
- **Total entradas = $8+3 = 11$ líneas de dirección** $\rightarrow 2^{11} = 2048 = 2K$ direcciones (256 caracteres $\times$ 8 columnas cada uno, exactamente, sin desperdicio).
- **Salida:** el patrón de esa columna $\rightarrow 8$ bits ($Y_0..Y_7$, uno por fila).
- **Tamaño: ROM de $2K \times 8$.**

Verificación de capacidad exacta: $256 \text{ caracteres} \times 8 \text{ columnas} = 2048 \text{ combinaciones} = 2^{11}$
 ✓ — la ROM se llena exactamente, sin direcciones sobrantes (a diferencia, por ejemplo, del Ejemplo BCD del Módulo 3 donde sobran direcciones).

(b) Construcción a partir de ROMs de 1K×8. Se necesita una ROM de 2K×8 y se dispone de chips de 1K×8 (mismo ancho de palabra $M=8$, solo cambia N) → caso **b) mayor cantidad de palabras** de la técnica estándar del módulo:

- Cantidad de chips = $2K/1K = 2 \text{ chips}$.
- Líneas extra = $\log_2(2048) - \log_2(1024) = 11 - 10 = 1 \text{ línea}$ (la más significativa de la dirección total, que en este diseño es justamente el bit más significativo del selector de columna) → decodificador 1→2 (inversor + AND, igual que en el Ejercicio 1.b) que activa el CS de uno de los 2 chips.
- Las 10 líneas restantes (carácter completo + 2 bits bajos de columna) van en paralelo a ambos chips.
- Las salidas $D_0..D_7$ de ambos chips se conectan directamente al mismo bus (solo un CS activo a la vez).

Verificación: $2 \times 1024 \times 8 = 16384 \text{ bits} = 2048 \times 8$ ✓ (coincide exactamente, sin sobrar capacidad).

Diseño alternativo de la solución de referencia del curso: Ejercicios

[Resueltos.pdf](#) resuelve este ejercicio de otra manera: direcciona la ROM **solo por el carácter** (8 bits → 256 direcciones) y saca **las 8 columnas a la vez** como 64 líneas de salida (8 columnas × 8 bits), construyéndola con **8 chips** de 1K×8 (uno por columna, cada uno almacenando la tabla completa de 256 caracteres para esa columna). Es un diseño técnicamente válido — cualquier fabricante podría construirlo así — pero no explota el direccionamiento “columna por columna” que sugiere el enunciado, usa 8 chips en vez de 2, y no ilustra la técnica central del módulo (combinar por cantidad de direcciones) tan directamente. Ambos diseños son defendibles; el de este documento se prefiere aquí por ser más económico (2 chips vs. 8) y por encajar mejor con la frase literal del enunciado y con el patrón ya usado en el Ejercicio 3.

Ejercicio 5 — ROM de 32K×16 como base

Chip disponible: **32K×16 bits**. $N=32K=2^{15}=32768$ (15 líneas de dirección), $M=16$ (16 líneas de datos).

(a) Construir una ROM de 64K×8 bits. Capacidad total necesaria = $65536 \times 8 = 524288$ bits. Capacidad del chip disponible = $32768 \times 16 = 524288$ bits — **exactamente igual**: no hace falta más capacidad física, solo **reorganizarla**. La idea es partir cada palabra de 16 bits del chip en **dos palabras “virtuales” de 8 bits** (mitad baja $Y_0..Y_7$, mitad alta $Y_8..Y_{15}$), lo que duplica la cantidad de direcciones “vistas desde afuera”: $2^{15} \times 2 = 2^{16} = 65536 \checkmark$.

- Las 15 líneas de dirección internas del chip ($A_0..A_{14}$) se conectan directamente a las 15 líneas de menor peso de la dirección externa.
- Se agrega **1 línea de dirección extra** (A_{15} , la nueva línea más significativa) que actúa como **selector de mitad**: mediante compuertas (un inversor + AND por cada bit de salida, o equivalentemente 8 multiplexores $2 \rightarrow 1$), $A_{15}=0$ conecta $D_i=Y_i$ (mitad baja) y $A_{15}=1$ conecta $D_i=Y_{i+8}$ (mitad alta), para $i=0..7$.
- CS y OE del chip 32K×16 se manejan igual que siempre (activos según corresponda al circuito externo).

Verificación: $2^{15} \times 16 = 2^{16} \times 8 = 524288$ bits $\checkmark$ (transformación exacta, sin desperdicio ni faltante).

(b) Sumador de 8 bits sin acarreo usando la ROM de 64K×8 construida en (a). Un sumador es, en sí mismo, una función lógica arbitraria de sus entradas — se puede implementar como traductor (Sección 4 de la guía temática): la ROM ya construida tiene exactamente **16 líneas de dirección y 8 líneas de datos**, que calzan perfectamente con “2 operandos de 8 bits $\rightarrow$ resultado de 8 bits (sin acarreo)”:

- **Entradas:** $A_0..A_7$ = operando 1 (8 bits); $A_8..A_{15}$ = operando 2 (8 bits). Total 16 líneas de dirección = $2^{16}=65536$ combinaciones, una por cada par de operandos posible.
- **Salidas:** $D_0..D_7$ = resultado de la suma módulo 256 (se descarta el acarreo de salida, como pide el enunciado).
- El circuito es directamente la ROM de 64K×8 de la parte (a), sin lógica adicional: cada dirección ya contiene, precargado, el resultado de sumar sus dos mitades.

(c) Contenido en la dirección 0x1794. Con la convención $A_0..A_7$ =operando1, $A_8..A_{15}$ =operando2:

- $0x1794 = 0001\ 0111\ 1001\ 0100$ (16 bits).
- Byte bajo ($A_0..A_7$) = $0x94 = 148$ (operando 1).
- Byte alto ($A_8..A_{15}$) = $0x17 = 23$ (operando 2).
- Suma módulo 256 = $148+23 = 171 = \mathbf{0xAB} = 10101011$ (binario).

Contenido a cargar en la dirección 0x1794: 0xAB.

(Nota: como la suma es conmutativa, el resultado 0xAB es el mismo sin importar cuál de los dos bytes se llame “operando 1” y cuál “operando 2” — la única convención que importa es cuál grupo de 8 bits de la dirección corresponde a cada operando, no el orden de la suma en sí.)

Ejercicio 6 — ROM traductora de punto flotante 16→32 bits (sin solución de referencia)

(a) Organización y tamaño. La transformación de un valor de punto flotante de media precisión (IEEE 754, 16 bits: 1 signo + 5 exponente + 10 mantisa) a simple precisión (32 bits: 1 signo + 8 exponente + 23 mantisa) depende únicamente de los 16 bits de entrada — es exactamente el esquema de “ROM como traductor”:

- **Entradas:** $A_0..A_{15}$ = los 16 bits del número en media precisión $\rightarrow 2^{16} = 65536 = 64K$ direcciones (cubre absolutamente todos los patrones de bits posibles de entrada, incluyendo casos especiales: cero, denormalizados, infinito, NaN).
- **Salidas:** $D_0..D_{31}$ = los 32 bits del número equivalente en simple precisión $\rightarrow 32$ líneas de datos.
- **Tamaño:** ROM de 64K×32 bits.

Verificación: $\log_2(65536)=16$ ✓; las direcciones de ejemplo de la parte (b), 0x08C3 y 0x803B, caben en 16 bits (rango 0x0000–0xFFFF) ✓, consistente con este tamaño.

(b) Contenido en las direcciones 0x08C3 y 0x803B.

Formato media precisión: bit 15 = signo, bits 14-10 = exponente (sesgo 15), bits 9-0 = mantisa (10 bits).

Dirección 0x08C3 = 0000 1000 1100 0011:

- signo = 0
- exponente = bits 14-10 = 00010 = 2 ($\neq 0$ y $\neq 31 \rightarrow$ número normalizado)
- mantisa = bits 9-0 = 0011000011 = 195
- Valor = $+2^{2-15} \times (1 + 195/1024) = 2^{-13} \times (1219/1024) \approx 1.45316 \times 10^{-4}$

Para pasar a simple precisión (sesgo 127): exponente sin sesgo se mantiene igual (−13, ya que el número está normalizado en ambos formatos) $\rightarrow$ exponente simple = $-13 + 127 = 114 =$

01110010. La mantisa de 10 bits se extiende a 23 bits agregando 13 ceros a la derecha (sin

necesidad de redondeo, porque el valor es exactamente representable): mantisa32 = `00110000110 0000000000` (10 bits originales + 13 ceros).

- **Contenido en 0x08C3:** signo=0, exponente= `01110010`, mantisa= `001100001100000000000000` → palabra completa = `0 01110010 001100001100000000000000` = **0x39186000**.

Dirección 0x803B = `1000 0000 0011 1011` :

- signo = 1
- exponente = bits 14-10 = `00000` = 0 → caso **denormalizado** (no hay “1” implícito; el exponente real es -14, fijo)
- mantisa = bits 9-0 = `0000111011` = 59
- Valor = $-(59/1024) \times 2^{-14} = -59 \times 2^{-24} \approx -3.5167 \times 10^{-6}$

Al pasar a simple precisión, este número denormalizado en media precisión **sí es representable como normalizado** en simple precisión (el rango de exponentes de simple precisión es mucho más amplio), así que hay que normalizarlo: $59 = 11011$ en binario (6 bits) = $1.11011_2 \times 2^5$. Entonces el valor es $1.11011_2 \times 2^{5-24} = 1.11011_2 \times 2^{-19}$.

- Exponente simple = $-19+127 = 108 = 01101100$.
- Mantisa simple (23 bits) = los bits después del “1.” (`11011`) más ceros de relleno = `110110000000000000000000`.
- **Contenido en 0x803B:** signo=1, exponente= `01101100`, mantisa= `110110000000000000000000` → palabra completa = `1 01101100 110110000000000000000000` = **0xB66C0000**.

Verificación independiente: ambos resultados fueron chequeados con `struct.pack('>f', valor)` en Python a partir del valor decimal calculado manualmente por descomposición de bits, y coinciden exactamente con el cálculo manual bit a bit (incluyendo el caso de normalización del denormalizado en 0x803B). Ver también el patrón de “extender mantisa con ceros a la derecha cuando el exponente ya es normalizado” — es análogo (aunque en dirección opuesta) a la idea de mantisa/exponente vista en el Módulo 1 (IEEE754).

(c) Circuito combinatorio con compuertas y sumadores de n bits (solo para el caso normalizado, sin ROM). Como el enunciado pide ahora una implementación **sin** ROM, para números ya normalizados (exponente media precisión entre 1 y 30):

1. **Signo:** se copia directo, $D_{31} = A_{15}$ (cable directo, sin lógica).

2. **Mantisa:** los 10 bits de la mantisa de entrada se copian directo a los 10 bits más significativos de la mantisa de salida (23 bits); los 13 bits menos significativos restantes de la mantisa de salida se fijan en 0 (sin conexión, a masa). No hace falta redondeo porque toda mantisa de 10 bits es exactamente representable con 23 bits.
3. **Exponente:** el exponente de media precisión (5 bits, sesgo 15) hay que convertirlo al exponente de simple precisión (8 bits, sesgo 127). Como ambos representan el mismo exponente real $e = \text{exponente}_{\text{media}} - 15 = \text{exponente}_{\text{simple}} - 127$, se despeja: $\text{exponente}_{\text{simple}} = \text{exponente}_{\text{media}} + (127 - 15) = \text{exponente}_{\text{media}} + 112$. Esto se implementa con un **sumador de n bits** ($n=8$, con el operando de 5 bits extendido con ceros a la izquierda) que suma la constante fija 112 (= 01110000) al exponente de entrada extendido — exactamente el sumador de n bits que el enunciado permite usar.
4. (Caso denormalizado, si se pidiera cubrir también ese caso sin ROM: haría falta además un **detector de la posición del primer bit en 1** de la mantisa — un “priority encoder” — y un desplazador (shifter) para normalizarla, más lógica adicional para ajustar el exponente según cuántas posiciones se desplazó. El enunciado de este ítem se limita a “normalizados”, así que ese circuito adicional no es necesario aquí.)

Ejercicio 7 — ROM de 8 bits maximizando capacidad a partir de 4 chips distintos (en OpenFing)

Chips disponibles: $2 \times (2K \times 4 \text{ bits})$, $1 \times (4K \times 8 \text{ bits})$, $1 \times (1K \times 16 \text{ bits})$.

(a) Tamaño total maximizando capacidad.

Capacidad de cada chip:

- $2K \times 4$: $2048 \times 4 = 8192$ bits cada uno $\rightarrow$ los 2 chips juntos: 16384 bits.
- $4K \times 8$: $4096 \times 8 = 32768$ bits.
- $1K \times 16$: $1024 \times 16 = 16384$ bits.

Capacidad total disponible = $16384 + 32768 + 16384 = \mathbf{65536 \text{ bits}}$ (2^{16}).

Como la ROM final debe tener 8 bits de ancho de palabra: cantidad de palabras = $65536 / 8 = \mathbf{8192 = 8K \text{ palabras}}$.

- **Entradas:** $A_0..A_{12}$ (13 líneas de dirección, $\log_2(8192)=13$).
- **Salidas:** $D_0..D_7$ (8 líneas de datos).
- **Tamaño: ROM de $8K \times 8$ bits = 8 KB.**

Verificación: $8192 \times 8 = 65536$ bits = capacidad total disponible ✓ (se aprovecha exactamente el 100% de la capacidad de los 4 chips, sin desperdicio — cumple “maximizar” de forma exacta, no solo aproximada).

(b) Circuito completo.

Los chips **no** son todos del mismo tamaño, así que no alcanza con el esquema uniforme de un único decodificador sobre bits más significativos (visto en los Ejercicios 1 y 4): hay que armar un **mapa de memoria** con regiones de distinto tamaño y, dentro de cada región, aplicar las técnicas estándar del módulo (paralelo para ancho, banco+decodificador para cantidad de palabras). Conviene alinear las regiones en fronteras de potencias de 2 para que la selección siga siendo un árbol de decodificación simple:

1. **Los 2 chips de $2K \times 4$** se combinan **en paralelo** (mismas 11 líneas de dirección, mismo CS, salidas concatenadas D_0 - D_3 y D_4 - D_7) → forman un bloque lógico de **$2K \times 8$** (2048 palabras).
2. **El chip de $1K \times 16$** se reorganiza con la misma técnica del Ejercicio 5.a (partir cada palabra de 16 bits en dos de 8 bits, agregando 1 línea de dirección extra que selecciona con compuertas cuál mitad de los 16 bits sale al bus) → se convierte en un bloque lógico de **$2K \times 8$** también ($1024 \times 2 = 2048$ palabras).
3. **El chip de $4K \times 8$** ya tiene el ancho correcto (8 bits) → se usa directo como bloque de **$4K \times 8$** (4096 palabras).
4. Los tres bloques resultantes ($2K + 2K + 4K = 8K$) se combinan con un **decodificador de 2 bits (A_{11}, A_{12}) sobre los bits más significativos** de la dirección de 13 bits, en forma de árbol:
 - $A_{12}=1$ → selecciona el bloque de $4K \times 8$ (usa $A_0..A_{11}$ directo hacia el chip de $4K \times 8$).
 - $A_{12}=0, A_{11}=0$ → selecciona el bloque de los 2 chips de $2K \times 4$ en paralelo (usa $A_0..A_{10}$).
 - $A_{12}=0, A_{11}=1$ → selecciona el bloque del chip de $1K \times 16$ reorganizado (usa $A_0..A_9$ como dirección interna del chip, y A_{10} como bit selector de mitad de palabra, con la lógica de compuertas del Ejercicio 5.a).
5. Las salidas $D_0..D_7$ de los tres bloques se unen directamente al mismo bus de 8 bits: como el decodificador garantiza que solo un bloque tiene su CS activo en cada instante, no hace falta OR cableado (evitando el problema de la parte (f) del cuestionario teórico).

Verificación de cobertura: $4K (A_{12}=1) + 2K (A_{12}=0, A_{11}=0) + 2K (A_{12}=0, A_{11}=1) = 8K$ direcciones, cubiertas exactamente una vez cada una, sin huecos ni solapamientos ✓.

Ejercicio 8 — ALU simulada con ROM (Examen Julio 2022, sin solución de referencia)

(a) Construir una ROM de $2^{18} \times 10$ a partir de la cantidad mínima de chips de $2^{15} \times 20$.

Solo cambia N (cantidad de palabras); el ancho disponible (20 bits) es mayor al necesario (10 bits), así que **sobra** ancho — no hace falta combinar en paralelo por ese eje, alcanza con usar 10 de las 20 salidas de cada chip.

- Cantidad de chips = $2^{18}/2^{15} = 2^3 = 8$ chips (cantidad mínima posible, ya que 2^{15} es el tamaño de chip disponible y 8 es exactamente lo que se necesita para cubrir 2^{18} direcciones sin excedente).
- Líneas extra = $18 - 15 = 3$ líneas (A_{15}, A_{16}, A_{17}) → decodificador $3 \rightarrow 8$, cuyas 8 salidas activan el CS de cada uno de los 8 chips.
- Las 15 líneas menos significativas ($A_0..A_{14}$) van en paralelo a los 8 chips (dirección interna de cada uno).
- De las 20 salidas de cada chip, se usan solo 10 (p. ej. $Y_0..Y_9$) conectadas al bus de datos $D_0..D_9$; las 10 restantes de cada chip quedan sin conectar.
- OE se deja activo en todos (o común): CS del decodificador ya garantiza que solo un chip impone su salida sobre el bus en cada instante.

Verificación: $2^{18} = 262144$, $2^{15} = 32768$, $262144/32768 = 8 = 2^3 \checkmark$. No es un problema que sobre ancho de palabra (a diferencia de si sobrara o faltara cantidad de palabras, que si generaría huecos de direccionamiento).

(b) Programa que carga el contenido para simular la ALU.

Distribución de bits de entrada (según enunciado): bit0-1 = operación, bit2-9 = operando A (8 bits), bit10-17 = operando B (8 bits). Salida: bit0 = Z, bit1 = overflow (solo op. 00), bit2-9 = resultado (8 bits).

```

void cargar_rom(unsigned short rom[262144]) { // 2^18 direcciones, 10 bits utiles c/u
    for (unsigned int addr = 0; addr < 262144; addr++) {
        unsigned char op = addr & 0x3;          // bits 0-1
        unsigned char A = (addr >> 2) & 0xFF; // bits 2-9
        unsigned char B = (addr >> 10) & 0xFF; // bits 10-17

        unsigned char resultado = 0;
        unsigned char overflow = 0;

        switch (op) {
            case 0: { // suma en complemento a 2
                resultado = (unsigned char)(A + B); // mod 256, sin acarreo de salida
                unsigned char signoA = A & 0x80;
                unsigned char signoB = B & 0x80;
                unsigned char signoR = resultado & 0x80;
                // overflow: operandos de igual signo y resultado de signo distinto
                overflow = (signoA == signoB) && (signoR != signoA);
                break;
            }
            case 1: // AND bit a bit
                resultado = A & B;
                break;
            case 2: // OR bit a bit
                resultado = A | B;
                break;
            case 3: // NOT A (ignora B)
                resultado = (unsigned char)(~A);
                break;
        }

        unsigned char Z = (resultado == 0) ? 1 : 0;

        // empaquetar salida de 10 bits: bit0=Z, bit1=overflow, bit2-9=resultado
        rom[addr] = (unsigned short)(Z | (overflow << 1) | (resultado << 2));
    }
}

```

Verificación de la lógica de overflow (suma en complemento a 2, 8 bits): overflow ocurre si y solo si ambos operandos tienen el mismo signo y el resultado tiene signo distinto (positivo+positivo=negativo, o negativo+negativo=positivo). Casos de control:

- A=0x7F(+127), B=0x01(+1) → suma=0x80(-128 en 8 bits): signoA=signoB=0, signoR=0x80≠0 → overflow=1 ✓ (127+1 no entra en 8 bits con signo).
- A=0x80(-128), B=0xFF(-1) → suma=0x7F(+127): signoA=signoB=0x80, signoR=0≠0x80 → overflow=1 ✓ (-128-1 tampoco entra).
- A=0x01, B=0x02 → suma=0x03: sin overflow ✓ (caso normal).

Nota: el índice `addr` del `for` cumple, igual que en el Ejercicio 2 y el patrón general del módulo, triple función: contador, dirección de la ROM y fuente directa de los campos de entrada (operación, A, B) vía máscaras y corrimientos — nunca una variable separada que pueda desincronizarse de la dirección real.

Resumen de técnicas usadas en este práctico

Ejercicio	Técnica
Teóricas (a)-(f)	Definiciones núcleo: CS/OE, combinatorio vs. secuencial, tercer estado, variantes de ROM, riesgo del OR cableado
1	Combinación por cantidad de palabras (a) y caso general N+M (b)
2	ROM como traductor (tabla de verdad completa), programa en C con índice=dirección=dato
3	ROM como traductor con dos campos de entrada concatenados (dato + selector); bug de precedencia <code>&</code> vs. <code>==</code> en C
4	Mismo patrón dato+selector que Ej. 3, aplicado a una figura; combinación por cantidad de palabras con chips más chicos
5	Reinterpretar una palabra ancha como dos palabras angostas (duplicar N usando M como “banco” con mux); ROM como función aritmética (sumador)
6	ROM como traductor de formato binario completo (IEEE754 media→simple precisión); conversión de sesgo de exponente y normalización de denormalizados; alternativa sin ROM con sumador de n bits
7	Mapa de memoria con regiones de tamaño desigual: árbol de decodificación + combinación de las técnicas de los Ejercicios 1 y 5
8	Combinación por cantidad de palabras con ancho sobrante; ROM como traductor de una función compleja (ALU) con múltiples campos de entrada/salida y lógica de overflow

Para el examen: según el índice de evaluaciones anteriores (`plan_de_estudio.md`), la **combinación de ROMs a partir de chips más chicos** aparece en 7 de 9 evaluaciones — es, con diferencia, el contenido más transferible del módulo. El patrón de estudio recomendado:

1. Identificar primero **qué eje cambia** (N, M, o ambos) antes de dibujar nada — es la fuente más común de error de tolerancia cero.

2. Recalcular siempre $\log_2(N)$ para las líneas de dirección, nunca $\log_2(N \times M)$.
3. Verificar al final con la cuenta de capacidad total en bits: (cantidad de chips) $\times$ (capacidad de cada chip) debe dar exactamente la capacidad pedida cuando los tamaños son potencias de 2 “prolijas”.
4. La otra mitad recurrente (6/13 exámenes según el mismo índice) es “ROM como traductor + programa en C”, frecuentemente cruzada con el Módulo 1 (BCD, IEEE754, como en este Ej. 6) o con diseño de ALU/FSM (como en este Ej. 8) — ahí la tolerancia cero pasa por no confundir el índice del bucle con la dirección de memoria, y por extraer correctamente los campos de la dirección con máscaras y corrimientos de bits que sumen exactamente el ancho total de la dirección (2+8+8=18 en el Ej. 8, 8+3=11 en los Ejercicios 3 y 4).